

We apply sec.gov APIs to extract board of director names for a specific company, which is our proximal endpoint.

1. The sec.gov API is detailed and extremely well documented, but like most government services the documentation is impenetrable, not entirely correct, and most likely varies over time. It really ought to be laid out in a user-friendly format.

All data at sec.gov is public and freely available; posting on the other hand involves strict requirements.

As I looked around for tools that could achieve our endpoint, it emerged that they are largely proprietary, require an API key, and incur usage charges. The latter is often not evident at first glance, and only once one has studied the API documentation in detail. I have the sense this is intentional. I don't know if there is any freeware for this purpose, and if not, I can see why not. If the documentation were as good as many of the APIs Josh and Bryan recommended, I'd guess that freeware would be readily available.

I used chatGPT not only for research about what my endpoint should be, and also for how to code and implement it. The code that chatGPT wrote does not work in my hands, most likely because the directory structure changed since the last time chatGPT's database was updated.

My primary effort has been invested in elucidating the directory structure of the sec.gov website and revising the code that chatGPT wrote to properly account for this directory structure. Although these revisions are not complete and the code works only partly, it is obvious that now that I've figured out the directory and file structure, I can get the code to work properly within a few days.

I've attached (or will attach) the chatGPT transcripts so that the reader understands what information chatGPT provided.

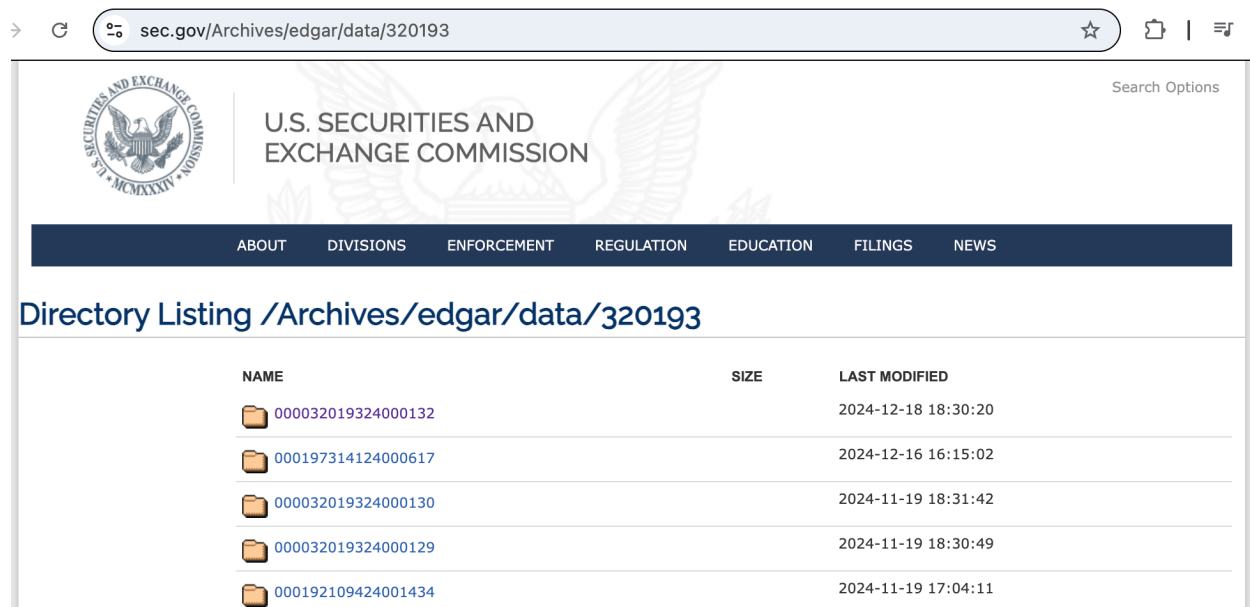
2. Directory and file structure:

https://www.sec.gov/	a home page
https://www.sec.gov/Archives	oops page not found
https://www.sec.gov/Archives/	URL /Archives/ not found on this server
https://www.sec.gov/Archives/edgar/	Not Found
https://www.sec.gov/Archives/edgar/data/	Not Found:

"The requested URL /Archives/edgar/data/ was not found on this server.

Additionally, a 404 Not Found error was encountered while trying to use an ErrorDocument to handle the request. *Apache Server at www.sec.gov Port 80*"

<https://www.sec.gov/Archives/edgar/data/320193> for cik = '0000320193" (Apple's CIK) yields a directory listing as seen below:



NAME	SIZE	LAST MODIFIED
 000032019324000132		2024-12-18 18:30:20
 000197314124000617		2024-12-16 16:15:02
 000032019324000130		2024-11-19 18:31:42
 000032019324000129		2024-11-19 18:30:49
 000192109424001434		2024-11-19 17:04:11

Within each of these “CIK subdirectories” such as for example, <https://www.sec.gov/Archives/edgar/data/320193/000130817925000009> found in <https://www.sec.gov/edgar/browse/?CIK=320193&owner=exclude>, we want the “defa14a” form submissions, e.g. “defa14a.htm” .

These apparently correspond to the “proxy statements” and/or “additional definitive proxy soliciting materials and Rule 14(a)(12) materials.” See for example: <https://www.sec.gov/edgar/browse/?CIK=320193&owner=exclude>

The defa14a.htm (proxy statements) are submitted a few times a year by SEC regulation and most CIK subdirectories do not contain one.

But some CIK subdirectories each year do contain a file such as: <https://www.sec.gov/Archives/edgar/data/320193/000130817925000009/aapl4359751-defa14a.htm> and in the [aapl4359751-defa14a.htm](#) file we find the Schedule 14A proxy statement that includes names of the members of the board of directors.

This last item can be parsed to extract board member names; all publicly held companies are required to submit this information, but how standard the format is unclear.

The example above appears to be too recent and the full report to shareholders is not included, but for an older proxy the full report can be found: <https://www.sec.gov/Archives/edgar/data/320193/000130817925000008/aapl4359751-def14a.htm>. The report includes board of director biographies, and other officials, and plenty of other

information. It can be parsed to obtain board member bios, and perhaps chief operating officer names.

Of course, this is Apple and not all shareholder reports are likely to be as nicely formatted.

In summary,

- given a company name;
- we obtain the CKC vi the SEC's form interface that we hijack;
- we check all the subdirectories of the CKC to search them for defa14a forms;
- If we find a defa14a form we extract the board of director names;
- If we find a shareholder report we can extract bios and names of executives.

These tasks can be accomplished by fetch/await, together with some html parsing.

The most recent list of board of directors for the given company is our endpoint.

Further processing on these lists is a data science/machine learning/artificial intelligence project.

ChatGPT 4o

Share

JN

You said:

how do I find the board members of apple inc.?

ChatGPT said:

ChatGPT

As of January 2025, Apple's Board of Directors comprises the following members:

- **Arthur D. Levinson:** Chairman of the Board; Founder and CEO of Calico.
- **Tim Cook:** CEO of Apple Inc.
- **Wanda Austin:** Former President and CEO of The Aerospace Corporation.
- **Alex Gorsky:** Former Executive Chair of Johnson & Johnson.
- **Andrea Jung:** President and CEO of Grameen America.
- **Monica Lozano:** Former President and CEO of College Futures Foundation.
- **Ronald D. Sugar:** Former Chair and CEO of Northrop Grumman Corporation.
- **Susan L. Wagner:** Co-founder and Director of BlackRock.

This information is available on Apple's official Leadership page.

[Apple](#)

In January 2024, Apple announced that former Vice President Al Gore and former Boeing CFO James Bell would retire from the board, effective February 2024. Subsequently, Wanda Austin was nominated to join the board.

[Fox Business](#)

For the most current information, you can visit Apple's official Leadership page.

[Apple](#)

Sources

40

You said:

Is this information available at data.sec.gov?

ChatGPT said:

ChatGPT

Yes, you can find information about the board members of Apple Inc. (or any publicly traded company) on the U.S. Securities and Exchange Commission's (SEC) website at data.sec.gov or the SEC's EDGAR database.

To locate information on Apple's board members via the SEC:

1. **Go to the EDGAR database**
(<https://www.sec.gov/edgar/searchedgar/companysearch.html>).
2. **Search for Apple Inc.:**
 - Use Apple's official company name, *Apple Inc.*, or its SEC Central Index Key (CIK): **0000320193**.
3. **Look for the Proxy Statement (DEF 14A):**
 - The proxy statement is filed annually and contains detailed information about the company's board of directors, executive compensation, and governance practices.
4. **Open the latest DEF 14A filing:**
 - Review the document for sections about board members. This will provide names, roles, biographies, and committee memberships.

Using the SEC filings ensures you're accessing official and up-to-date information directly from the company's regulatory submissions.

40

You said:

I see the proxy statements. How do I extract the names of board members and officers from these statements?

ChatGPT said:

ChatGPT

To extract the names of board members and officers from proxy statements (DEF 14A) efficiently, you can follow these steps:

1. Open the Proxy Statement

- Access the DEF 14A document from the SEC's EDGAR database.
 - Open the filing and look for sections titled:
 - "Board of Directors"
 - "Corporate Governance"
 - "Executive Officers and Directors"
 - "Election of Directors" (usually near the beginning).
-

2. Search for Relevant Sections

Use the search function (Ctrl+F or Cmd+F) to quickly locate:

- **Board Members:** Search for keywords like "Director," "Board of Directors," or "Nominees for Election."
 - **Executive Officers:** Search for "Executive Officers," "Management," or "Compensation Discussion and Analysis."
-

3. Identify Names and Roles

- Look for tables or lists that summarize the information.
 - Extract:
 - **Names** of directors and officers.
 - **Roles** (e.g., Chair, CEO, CFO).
 - Committee memberships (e.g., Audit, Compensation, Nominating Committees).
-

4. Copy and Organize Information

- Copy the relevant names and roles into a structured format such as a spreadsheet for easy reference.
-

5. Automate Extraction (Optional)

If you are handling multiple proxy statements or want to streamline the process:

- **Use Text Parsing Tools:**
 1. Tools like Python (with libraries such as `pdfplumber`, `PyPDF2`, or `BeautifulSoup` for HTML filings) can automate the extraction.
 - **Steps for Automation:**
 1. Download the proxy statement as a PDF or HTML file.
 2. Use a script to extract sections containing keywords like "Board of Directors" or "Executive Officers."
 3. Parse the text to isolate names, roles, and related information.
-

Example Python Code

Here's a basic example of extracting names and roles from a proxy statement:

python

Copy code

```
import re

# Load the proxy statement text
with open('proxy_statement.txt', 'r') as file:
    text = file.read()

# Search for board member and officer information
board_section = re.findall(r"Board of Directors.*?\n\n(.*?)\n\n",
text, re.DOTALL)
executive_section = re.findall(r"Executive
Officers.*?\n\n(.*?)\n\n", text, re.DOTALL)

# Extract names (example pattern, adjust based on the document
structure)
names = re.findall(r"[A-Z][a-z]+ [A-Z][a-z]+", text)

# Display results
print("Board Members and Officers:")
for name in names:
    print(name)
```

This approach allows you to handle documents programmatically and efficiently. If you need help implementing automation, feel free to ask!

40

You said:

give me javascript code to do this

ChatGPT said:

ChatGPT

Here's an example of JavaScript code that extracts names of board members and officers from a proxy statement in text format. It uses regular expressions and is designed for text files or HTML content:

javascript

Copy code

```
// Function to extract board members and officers from proxy statement text
function extractBoardMembersAndOfficers(proxyText) {
    // Regular expressions to find board member and officer sections
    const boardSectionRegex = /Board of Directors([\s\S]*?)\n\n/g;
    const executiveSectionRegex = /Executive Officers([\s\S]*?)\n\n/g;

    // Match and extract relevant sections
    const boardSectionMatch = proxyText.match(boardSectionRegex);
    const executiveSectionMatch =
proxyText.match(executiveSectionRegex);

    // Combine sections for processing
    const combinedSections = (boardSectionMatch ||
[]).concat(executiveSectionMatch || []).join('\n');

    // Regular expression to extract names (simple format for demonstration)
    const nameRegex =
/\b[A-Z][a-z]+\s[A-Z][a-z]+(?:\s[A-Z][a-z]+)?\b/g;

    // Extract names from the combined text
```

```
        const names = [...new Set(combinedSections.match(nameRegex))]; //
Use Set to remove duplicates

        return names;
    }

    // Example usage with a sample proxy statement text
    const proxyStatementText = `
Board of Directors
John Doe - Chairman
Jane Smith - CEO
Mark Johnson - Director

Executive Officers
Anna White - CFO
Michael Brown - COO
`;

    // Extract names
    const names = extractBoardMembersAndOfficers(proxyStatementText);

    // Display results
    console.log('Board Members and Officers:', names);
```

Steps to Use

- 1. Prepare the Proxy Statement:**
 - Ensure the proxy statement is in plain text or HTML format.
 - If the file is in PDF, convert it to text using a tool like [pdf2txt](#).
- 2. Run the Code:**
 - Copy the proxy statement content into the `proxyStatementText` variable or load it dynamically from a file.
 - Run the script in a Node.js environment or in the browser (if loading the text dynamically).
- 3. Extend the Regex (if needed):**
 - Modify the `nameRegex` to better suit the document's naming conventions or format.

Output

The script will extract names such as:

less

Copy code

```
Board Members and Officers: ['John Doe', 'Jane Smith', 'Mark Johnson',  
'Anna White', 'Michael Brown']
```

Let me know if you need help customizing this further!

4o

Is this conversation helpful so far?

O

ChatGPT can make mistakes. Check important info.

?

ChatGPT 4o

Share

JN

You said:

js list files in directory "dir"

ChatGPT said:

ChatGPT

To list files in a directory in JavaScript, you can use Node.js's built-in `fs` module. Here's an example:

javascript

Copy code

```
const fs = require('fs');  
const path = require('path');  
  
// Define the directory  
const directoryPath = path.join(__dirname, 'dir');  
  
// Read files from the directory  
fs.readdir(directoryPath, (err, files) => {  
  if (err) {  
    console.error('Error reading directory:', err);  
    return;  
  }  
})
```

```
console.log('Files in directory:');
files.forEach(file => {
  console.log(file);
});
});
```

Explanation:

1. **fs.readdir**: Asynchronously reads the contents of the directory.
2. **path.join**: Ensures the directory path is constructed correctly, regardless of the operating system.
3. **Error handling**: The callback checks if there was an error, like if the directory doesn't exist.
4. **files**: The **files** array contains the names of all files and folders in the directory.

If you want only files (excluding subdirectories), you can add a filter using **fs.stat** or **fs.promises.stat** to check if an item is a file:

javascript

Copy code

```
const fs = require('fs');
const path = require('path');

const directoryPath = path.join(__dirname, 'dir');

fs.readdir(directoryPath, (err, files) => {
  if (err) {
    console.error('Error reading directory:', err);
    return;
  }

  console.log('Files only:');
  files.forEach(file => {
    const filePath = path.join(directoryPath, file);
    fs.stat(filePath, (err, stats) => {
      if (err) {
        console.error('Error getting stats:', err);
        return;
      }
    });
  });
});
```

```
    }

    if (stats.isFile()) {
        console.log(file);
    }
});
});
});
```

For Promise-based Approach

Use `fs.promises` for cleaner, async/await-style code:

javascript

Copy code

```
const fs = require('fs').promises;
const path = require('path');

const listFiles = async (directoryPath) => {
    try {
        const files = await fs.readdir(directoryPath);
        console.log('Files in directory:');
        for (const file of files) {
            const filePath = path.join(directoryPath, file);
            const stats = await fs.stat(filePath);
            if (stats.isFile()) {
                console.log(file);
            }
        }
    } catch (err) {
        console.error('Error:', err);
    }
};

listFiles(path.join(__dirname, 'dir'));
```

These methods let you list and filter files in the directory efficiently.

You said:

```
javascript list the files in this URL: const filingUrl =
`${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/
-/g, ' ')}`;
```

ChatGPT said:

ChatGPT

To list files in a URL (such as a web directory), JavaScript alone cannot do this directly. Most web servers do not allow directory listing for security reasons. However, if the server supports directory listing and provides an HTML page or an API with the file listing, you can retrieve and parse it using JavaScript.

Steps:

1. Fetch the contents of the URL.
2. Parse the response to extract file names if directory listing is enabled.
3. Handle errors if the server does not expose the directory listing.

Here's an example using `fetch` to get the contents of a URL and extract file names:

Example Code:

javascript

Copy code

```
const SEC_BASE_URL = "https://www.sec.gov";
const cik = "0000320193"; // Example CIK
const accessionNumber = "0000320193-23-000052"; // Example accession
number
const filingUrl =
`${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(
/-/g, ' ')}`;

const listFilesInUrl = async (url) => {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }
    const text = await response.text();
```

```

    // Parse file names from HTML response
    const fileNames = [];
    const regex = /<a href="([^\"]+)"/g; // Matches href links in the
directory
    let match;
    while ((match = regex.exec(text)) !== null) {
        fileNames.push(match[1]);
    }

    console.log('Files in directory:');
    fileNames.forEach(file => console.log(file));
} catch (error) {
    console.error('Error fetching or parsing directory:', error);
}
};

listFilesInUrl(filingUrl);

```

Explanation:

1. **fetch**: Used to retrieve the content of the URL.
2. **Regex (<a href="([^\"]+)")**: Extracts links from the HTML response, assuming the server provides an HTML directory listing.
3. **Directory Listing**: Many servers disable directory listing, so this approach will only work if the server explicitly allows it.

Limitations:

- If the server does not support directory listing, this will not work.
- In such cases, you need to check if the server offers an API (like a JSON response) for accessing file details.

If you're dealing with the SEC's EDGAR system, it typically doesn't provide directory listings in this way. You might need to analyze the structure of their pages or use their official API for structured access to the data.

4o

O

ChatGPT can make mistakes. Check important info.

?